SHEHZAD NISAR

Junior Java Backend & Full-Stack Developer – Java, Spring Boot & React

Karachi, Pakistan | +92 344 3380292 | shehzadnisar07@gmail.com | linkedin.com/in/shehzadnisar07 | github.com/shehzadnisar07

PROFESSIONAL SUMMARY

Final-year Bachelor of Computer Science student at the University of Karachi building backend microservices and full-stack web applications with Java and Spring Boot on the backend and React with TypeScript on the frontend. Designed and built TransMoney, a six-service digital wallet platform using Spring Data JPA (Hibernate), Spring Security (JWT), Apache Kafka, Redis, and MySQL; seeking an entry-level Java Backend or Full-Stack Developer role.

TECHNICAL SKILLS

Languages: Java (Java 17, OOP, Collections Framework, Java Streams, Lambdas, Optionals, Custom Exceptions), SQL, TypeScript, JavaScript

Backend: Spring Boot, Spring Cloud Gateway, Spring Security, Spring Data JPA, Hibernate, OpenFeign, REST APIs, JWT, BCrypt, Jakarta Validation

Databases & Messaging: MySQL, Relational Data Modeling, Redis (Caching, Distributed Counters & Rate Limiting), Apache Kafka (Event-Driven Architecture, SAGA Pattern)

Frontend: React 19, TypeScript, React Router, Zustand, React Hook Form, Zod, Axios, Tailwind CSS, HTML5, CSS3

Tools & DevOps: Maven, Git, GitHub, Docker, Docker Compose, Vite, Postman, IntelliJ IDEA

PROJECTS

TransMoney – Microservices-Based Digital Wallet

Java 17, Spring Boot, Spring Cloud Gateway, Spring Security, Spring Data JPA (Hibernate), Kafka, Redis, MySQL, React 19, TypeScript, Docker Compose | *github.com/shehzadnisar07/trans-money-microservices*

- Built a six-service Spring Boot microservices system (API Gateway, Auth, Account, Transaction, Fraud Detection, Notification) exposing 11+ REST endpoints, communicating via synchronous OpenFeign clients and asynchronous Apache Kafka topics (transaction.initiated, transaction.completed, transaction.refunded, fraud.detected).
- Modeled relational database schemas in MySQL 8 (users, accounts, transactions) using Spring Data JPA and Hibernate (@Entity, UUID/Identity generation, enum mappings, and BigDecimal scale-2 precision), with SecureRandom 12-digit account number generation and account-type daily limits.
- Implemented stateless JWT authentication with Spring Security and BCrypt, configuring Spring Cloud Gateway filters for token verification, X-User-Email header rewriting to prevent identity spoofing, CORS policies, and Redis token-bucket rate limiting (10 requests/sec, burst of 20) on account and transaction routes.
- Developed SAGA-pattern money transfers with UUID idempotency keys: synchronously debiting sender balances via OpenFeign, tracking six lifecycle states (PENDING, PROCESSING, PENDING_VERIFICATION, COMPLETED, FAILED, FLAGGED), and executing compensating refunds on fraud or expired/invalid OTPs.
- Wrote a three-rule fraud detection service using Redis and BigDecimal arithmetic (60-second transaction velocity counters, running-average amount spikes, and balance-share thresholds) paired with Redis-backed time-limited OTP verification and automated account blocking.
- Built the React 19 + TypeScript frontend (Dashboard, Transfer, OTP Verification, Transaction History, Account Creation, and Notifications pages) using React Router v6, Zustand persistent state, React Hook Form, Zod schema validation, Axios JWT interceptors, and Tailwind CSS.

Console-Based Java Banking Application

Java, OOP, Collections (HashMap, ArrayList), Java Streams, Lambdas, Optionals, Custom Exceptions | *github.com/shehzadnisar07/Java-programming-backend*

- Built a four-layer console banking application (Domain, Repository, Service, and Validation layers) supporting six core operations: opening SAVING/CURRENT accounts with formatted sequential IDs (AC000001) and UUID customer keys, deposits, withdrawals, peer-to-peer transfers (TRANSFER_IN/TRANSFER_OUT), account statements, and customer search.
- Implemented in-memory repositories using Java Collections (HashMap and ArrayList with computeIfAbsent for per-account transaction ledgers) and used Java Streams (filter, flatMap, Comparator.comparing) and Optional (orElseThrow) to query customer accounts and sort chronological statements.
- Designed a generic @FunctionalInterface (Validation<T>) with lambda expressions and custom exceptions (AccountNotFoundException, InsufficientAmountException, ValidationException) to guard against invalid inputs, self-transfers, and insufficient-funds errors.

EDUCATION

Bachelor of Science in Computer Science 01/2023 – Expected 12/2026

University of Karachi, Karachi, Pakistan